

Technology Decision Reference

One page per technology. The mechanism that drives the tradeoff, when to reach for it, when it flips, and the line to say in the room.

This sheet answers "which technology and why" — the axis your System Design sheet doesn't cover (it's organized by pattern). Every page is built to the same skeleton so it's scannable under pressure. The **interview line** is the choosing-not-pattern-matching signal; the **pushback** is where the obvious answer reverses, which is the staff tell.

1	PostgreSQL — the default; ACID, joins, one node does more than you think
2	Cassandra — query-first, masterless, multi-region writes; you give up joins
3	Redis — in-memory speed; cache, counter, lock, queue, ephemeral state
4	Kafka — durable ordered log; decouple, buffer, replay, fan-out
5	S3 / Blob Storage — infinite cheap bytes; the bytes never touch your API
6	CDN — push bytes to the edge; the answer to "global" and "read-heavy static"
7	Vector DB — ANN over embeddings; the retrieval half of RAG
8	Workflow Orchestrator — durable fan-out/fan-in, exactly-once continuation
9	Elasticsearch — inverted index; full-text + faceted search, not a primary store

1 · PostgreSQL

The correct default. Reach for something else only when you can name the specific thing Postgres can't do at your scale.

MECHANISM

B-tree indexes, in-place updates, single-leader replication. One primary takes writes; read replicas stream the WAL (write-ahead log) asynchronously. MVCC gives readers a snapshot without blocking writers — each row can have multiple versions; VACUUM reclaims dead ones.

Writes do in-place page updates and page splits — random I/O, which is the ceiling vs. LSM stores.

REACH FOR IT WHEN

- You need **ACID transactions** across rows/tables — anything money, inventory, or multi-entity publish.
- Read patterns are **ad-hoc or unpredictable** — the planner + indexes handle new WHERE clauses for free.
- You need joins, aggregations, secondary indexes.
- You want **consistency by default**, not eventual.
- Scale is anything below "genuinely global + huge." Which is almost always.

AVOID / AUGMENT WHEN

- Write volume genuinely exceeds one primary (10k+ sustained writes/s to one logical table) **SHARD OR LSM STORE**
- You need active-active **multi-region writes** — retrofitted, painful.
- Append-only firehose (metrics, events) — LSM fits better.

NUMBERS TO ANCHOR

Point read (indexed)	< 1 ms
Writes/s, one primary	~5k–15k
Rows before sharding hurts	~10–100M+
Serialized updates/hot row	~500–1k/s
Read scaling	replicas, ~linear

INTERVIEW LINE

I default to Postgres and make something else justify itself. It gives me transactions, joins, and query flexibility for free, one primary does 5–15k writes/s, and reads scale on replicas — I only move off it when I can name the exact property it can't provide at my scale.

PUSHBACK / WHEN IT FLIPS

"Postgres doesn't scale" is usually wrong — it's that a **single hot row** or a **multi-region write** requirement flips it, not raw size. Sharded Postgres by a good key gets you most of Cassandra's write distribution; what you lose is automatic rebalancing and painless geo-replication, not the sharding itself.

2 · Cassandra

Query-first, masterless, born multi-region. You trade joins and ad-hoc queries for cheap global replication and painless rebalancing.

MECHANISM

LSM tree + consistent hashing + masterless replication. Writes append to a memtable + commitlog, flush to immutable SSTables — sequential I/O, so the write ceiling is high. Compaction merges SSTables in the background.

Consistent hashing (with vnodes) places partitions on a ring — adding a node auto-rebalances token ranges. Any replica takes writes; consistency is tunable per query (ONE, QUORUM, ALL). Multi-datacenter replication is a config declaration.

REACH FOR IT WHEN

- Access is **known, key-based lookups** — "everything for this id," "this partition sorted by time."
- Global reads needing **local-region latency** — multi-DC replication out of the box.
- Very high write throughput, append-heavy.
- You want capacity-by-adding-nodes with no manual reshard.

THE MODEL TAX (KNOW COLD)

- **No joins, no ad-hoc queries.** Query by non-partition column = full scan.
- **One table per query** — denormalize, dual-write, own consistency across copies yourself.
- Eventual consistency by default; LWT (Paxos) exists but is slow.
- Incomplete clustering key **silently upserts** (clobbers rows).

NUMBERS TO ANCHOR

Write throughput	very high, scales out
Single-partition read	low ms
Add capacity	join a node
Consistency	tunable per query

INTERVIEW LINE

Cassandra fits keyed lookups read globally and written rarely. Its edge over sharded Postgres isn't write throughput at my volume — it's that consistent hashing makes rebalancing automatic and multi-region replication a config line, not a project. I accept no joins and eventual consistency, which cost nothing for this access pattern.

PUSHBACK / WHEN IT FLIPS

Below genuine global scale, **Postgres is the better default** even for keyed data — you keep transactions and query flexibility and defer the operational tax (repair, compaction, eventual-consistency reasoning). Cassandra is right when scale × geo-distribution × access-pattern-simplicity all hold at once. State those conditions and you're choosing, not pattern-matching.

3 · Redis

In-memory, single-threaded, microsecond ops. A Swiss-army knife: cache, counter, lock, rate limiter, ephemeral queue, leaderboard. Rarely your source of truth.

MECHANISM

RAM-resident data structures, single-threaded command loop.

Single-threaded is a feature — every command is atomic, no locks. Lua scripts run atomically too (check-and-mutate in one shot).

Durability is optional: RDB snapshots + AOF append log (everysec is the usual). Replicas + Sentinel/Cluster for HA. Treat it as a fast cache that **can** persist, not a database that happens to be fast.

REACH FOR IT WHEN

- **Cache** in front of a slower store (the default use).
- **Atomic counters** — INCR/DECR for rate limits, inventory admission, fan-in completion.
- **Distributed lock** (short-lived; SET NX PX + fencing token).
- **ZSET** for leaderboards, waiting-room ordering, timer queues.
- Ephemeral state with **TTL auto-expiry** (sessions, holds).

AVOID / CAREFUL WHEN

- It's your **only** copy of important data — a failover to a lagging replica loses recent writes.
- Dataset > RAM. It's memory-bound; that's the cost model.
- TTL as a business trigger: expiry **deletes a key**, it doesn't run your compensation. Keyspace notifications are at-most-once — don't rely on them for correctness.

NUMBERS TO ANCHOR

Op latency	~0.1–0.5 ms
Throughput, one node	~100k+ ops/s
Bound	RAM + network

INTERVIEW LINE

I reach for Redis when I need one thing very fast and can tolerate losing it: a cache, an atomic counter, a TTL'd hold. Single-threaded means every op and every Lua script is atomic, which is why it's the right place for admission control and rate limiting. I keep the durable truth in Postgres behind it.

PUSHBACK / WHEN IT FLIPS

For a hold/expiry pattern, Redis TTL only wins if the **key IS the hold** (its disappearance is the release). If Postgres owns the counter, a DB sweeper beats Redis notifications — durable and self-healing. Use Redis for the ticket-onsale scale (100k holds/s, visible countdown); keep the sweeper for the hotel scale.

4 · Kafka

A durable, ordered, replayable log. Not a queue you drain — a commit log consumers read at their own offset. The backbone for decoupling and buffering.

MECHANISM

Partitioned append-only log; consumers track offsets. Each topic splits into partitions; order is guaranteed **within** a partition, not across. Messages persist for a retention window — consumers can replay by rewinding their offset. Producers key messages to control partition (same key → same partition → ordered).

Delivery is **at-least-once** by default; consumers must be idempotent. Exactly-once exists (txns) but adds cost.

REACH FOR IT WHEN

- **Decouple** producers from consumers — upload service emits, transcode fleet consumes independently.
- **Buffer a firehose** — absorb spikes, consumers process at their own rate (load leveling).
- **Fan-out** one event to many consumer groups (analytics + search index + notifications).
- **Event sourcing / CDC** — the log is the source of truth; replay to rebuild.

AVOID / CAREFUL WHEN

- You need **per-message ack/delete** or priority — that's a task queue (SQS/RabbitMQ), not a log.
- You need **global ordering** — you only get per-partition order.
- Low-volume simple job dispatch — Kafka is operational weight you may not need.

NUMBERS TO ANCHOR

Throughput	very high (MB/s+/partition)
Ordering	per-partition only
Delivery	≥ once (make idempotent)
Retention	hours→forever, configurable

INTERVIEW LINE

I use Kafka to decouple and buffer: producers append, consumer groups read independently at their own offset, and the log's retention lets me replay to rebuild a downstream or add a new consumer. I key by entity for per-partition ordering and make consumers idempotent because delivery is at-least-once.

PUSHBACK / WHEN IT FLIPS

Reaching for Kafka on a simple job queue is over-engineering — if you want per-message ack, retry, and a DLQ without replay or fan-out, **SQS/RabbitMQ is simpler**. Kafka earns its operational cost when you need replay, multiple independent consumers, or firehose buffering — name which one.

5 · S3 / Blob Storage

Effectively infinite, cheap, durable bytes over HTTP. The rule: large binaries live here, your DB stores the pointer, and the bytes never transit your API tier.

MECHANISM

Key → object store over HTTP, 11-nines durability. Not a filesystem — flat keypace, objects are immutable blobs (overwrite, don't edit). Durability comes from replication across AZs. Tiered storage (hot → infrequent → archive) trades retrieval latency for cost.

Presigned URLs let clients read/write directly with time-limited credentials — your server signs, the client transfers. **Multipart upload** splits large objects into parts (parallel, resumable).

REACH FOR IT WHEN

- Any **large binary**: video, images, uploads, backups, ML artifacts, logs.
- Static assets to serve via CDN (S3 as origin).
- Data-lake / event archive (cheap, queryable later).
- Decoupling upload transfer from your app servers entirely.

THE TWO PATTERNS TO SAY

- Presigned direct upload** — client PUTs straight to S3; API only issues URLs. Bytes bypass your tier.
- Multipart** for big files — one presigned URL per part, parallel, per-part retry, resumable via ListParts.

NUMBERS TO ANCHOR

Durability	~11 nines
Single PUT cap (S3)	5 GB
Multipart part min / max	5 MB / 10k parts
First-byte latency	tens of ms

INTERVIEW LINE

Large binaries go to blob storage and the DB keeps only the key. Clients upload via presigned URLs so gigabytes never touch my API tier, and anything big uses multipart — one URL per part, parallel, resumable. I front it with a CDN for reads and drive state transitions off object-created events rather than trusting the client's "done."

PUSHBACK / WHEN IT FLIPS

S3 is not low-latency and not a database — no transactions, no querying inside objects, eventual consistency on some ops historically. Don't store data you'll filter/join on as blobs. And it's the **pointer-in-DB** pattern: an object with no DB row is an orphan you pay for — reconcile.

6 · CDN

Cache bytes at edge PoPs close to users. The answer to "global users" and "read-heavy static/cacheable content." Turns 20k QPS for one object into ~one origin fetch per PoP.

MECHANISM

Geographically distributed edge caches. User hits the nearest PoP; on a cache hit it serves locally (low latency, zero origin load). On a miss it fetches from origin, caches per TTL/headers, serves. Cache-Control / ETag govern freshness.

Collapses read fan-out: N users pulling the same object = ~1 origin fetch per PoP per TTL. Also absorbs traffic spikes and DDoS at the edge.

REACH FOR IT WHEN

- **Static assets:** JS/CSS/images, video segments, downloads.
- **Global read audience** for the same content.
- A **hot cacheable value** read enormously — e.g. the waiting-room "admitted" counter cached 2s collapses 500k polls to near-zero origin.
- Shielding origin from spikes.

AVOID / CAREFUL WHEN

- **Personalized / per-user** responses — low hit rate, little benefit (edge compute is the escape hatch).
- Highly dynamic data where staleness is unacceptable.
- Invalidation is the hard part — **short TTL** (let it expire) beats active purge where you can tolerate seconds of staleness.

NUMBERS TO ANCHOR

Edge hit latency	single-digit→tens ms
Origin offload	often > 95%
Invalidation	TTL > active purge

INTERVIEW LINE

A CDN pushes cacheable bytes to edge PoPs so global reads hit a nearby cache instead of my origin — that's how one popular object survives 20k QPS at ~one origin fetch per PoP. I lean on short TTLs over active invalidation whenever seconds of staleness are acceptable, which is most static and near-static content.

PUSHBACK / WHEN IT FLIPS

A CDN does nothing for **personalized or write** paths — don't wave it at a dynamic problem. The real design question is always **invalidation**: reason about what's cacheable and for how long, not just "add a CDN." Staleness tolerance is the actual variable.

7 · Vector Database

Approximate nearest-neighbor search over embedding vectors. The retrieval half of RAG: "find the *k* chunks most semantically similar to this query."

MECHANISM

ANN index over high-dim vectors. Text/images → embeddings (e.g. 768–3072 dims) via a model. Query is embedded the same way; the DB finds nearest neighbors by cosine/dot distance. Exact search is O(N) per query, so it uses an **approximate** index — usually HNSW (navigable small-world graph): logarithmic-ish search, tunable recall vs. latency.

Metadata filtering (tenant, date, ACL) runs alongside vector search — critical and easy to get wrong.

REACH FOR IT WHEN

- **RAG**: ground an LLM in your corpus — retrieve top-k chunks, stuff into context.
- Semantic search (meaning, not keywords).
- Recommendations / dedup / similarity by embedding.

PIPELINE TO SAY

Ingest: chunk → embed → upsert with metadata. Query: embed → ANN top-k → filter by ACL → rerank → into prompt. **Chunking strategy and reranking** move quality more than the DB choice.

AVOID / CAREFUL WHEN

- Keyword/exact-match is what users want — **BM25/Elasticsearch** may beat or hybridize with vectors.
- Small corpus — a library (FAISS) or **pgvector** in Postgres beats a new system.
- **Permission leakage**: embeddings ignore ACLs. Filter by permission at query time or you retrieve across tenants.

NUMBERS TO ANCHOR

Embedding dims	~768–3072
Vector size	dims × 4 bytes
1M × 1536-dim	~6 GB raw
Index	HNSW (recall↔latency)

INTERVIEW LINE

For RAG I embed and index chunks in a vector DB with HNSW, then at query time embed the question, pull top-k by cosine, filter by the user's ACL, and rerank before building the prompt. Chunking and reranking drive quality more than which vector DB. Below a few million vectors I'd just use pgvector and not add a system.

PUSHBACK / WHEN IT FLIPS

Vector search isn't always the answer — **hybrid** (BM25 + vector) often wins because pure semantic misses exact terms (names, IDs, codes). And the sharp failure mode is **permissions**: sensitive data in embeddings/logs and cross-tenant retrieval. Raise ACL-at-query-time unprompted — it's the People-Innovation kill-shot.

8 · Workflow Orchestrator

Temporal / Step Functions. Durable multi-step workflows with fan-out/fan-in and exactly-once continuation. The answer to "coordinate N async tasks and know when all are done."

MECHANISM

Event-sourced state machine + replay. Every state transition is appended to a durable history log — that log is truth, not in-memory state. On crash, the workflow re-executes from the top, but completed steps return their **recorded results** from the log instead of re-running; real execution resumes only past the end of history.

Fan-in is a read over the ordered log ("is there a completion for every scheduled task"), decided by a **single evaluator** — so no concurrent-counter race. Requires **deterministic** workflow code (no clocks/random outside activities).

REACH FOR IT WHEN

- **Fan-out/fan-in:** spawn N tasks, run one step when all finish (transcode ladder → mark READY).
- Long-running, multi-step, must survive crashes (order saga, provisioning).
- Steps need **retries, timeouts, compensation** without hand-rolled state.
- You'd otherwise build a status table + counter + reaper by hand.

AVOID / CAREFUL WHEN

- Single-step / fire-and-forget — a queue is enough.
- Ultra-low-latency synchronous paths (replay + persistence add overhead).
- **Activities are at-least-once** — the continuation is exactly-once, but activity code must be idempotent.

TWO FLAVORS

Temporal	your code + replay
Step Functions	declarative JSON FSM
Fan-in	built-in primitive
Guarantee	exactly-once continuation

INTERVIEW LINE

When I need to fan out N tasks and fire one step when all complete, I reach for an orchestrator instead of a hand-rolled counter-and-reaper. It persists every transition to a durable log and decides completion with a single reader over that ordered log, so the fan-in race is gone by construction. Continuation is exactly-once; I still make activities idempotent.

PUSHBACK / WHEN IT FLIPS

Don't reach for it for a single async job — that's a queue, and the orchestrator's replay/determinism model is real cognitive + operational cost. The mechanism to **name** even if you hand-roll: atomic DECR for "am I last," a guarded status transition for idempotent completion, and a reaper for stalls.

9 · Elasticsearch

An inverted index for full-text and faceted search. A secondary read model you feed from your primary store — not your source of truth.

MECHANISM

Inverted index (Lucene) + distributed shards. Text is analyzed (tokenized, lowercased, stemmed) into terms; the inverted index maps term → documents, so full-text queries are fast. Relevance ranked by BM25. Indexes shard + replicate for scale and HA.

Near-real-time, not immediate — a refresh interval (default ~1s) gates visibility. Async-fed from your primary DB via CDC or dual-write.

REACH FOR IT WHEN

- **Full-text search** with relevance ranking (search bars, docs, logs).
- **Faceted / filtered** search — aggregations across many fields (e-commerce filters).
- Log / observability search at scale (the ELK stack).
- Hybrid with vectors for semantic + keyword.

AVOID / CAREFUL WHEN

- As a **primary store** — it's a derived index; rebuild from truth. Don't put data only here.
- You need **strong consistency / transactions** — near-real-time and no ACID.
- Simple exact lookups — a DB index is cheaper and simpler.
- Keeping it in sync is the real cost — CDC pipeline + reconciliation.

NUMBERS TO ANCHOR

Visibility	near-real-time (~1s)
Ranking	BM25
Role	derived read model

INTERVIEW LINE

Full-text and faceted search go to Elasticsearch as a secondary read model fed from my primary store via CDC — the inverted index and BM25 give relevance ranking a B-tree can't. It's near-real-time and not my source of truth, so I design the sync pipeline and a reconciliation path, and I can rebuild the index from the primary at any time.

PUSHBACK / WHEN IT FLIPS

The trap is treating it as a database. It's a **derived index** — the hard part is the sync pipeline and its consistency, not the query. For semantic search, **hybrid BM25 + vector** usually beats either alone. For plain exact-match, a Postgres index is simpler and consistent.